

中間発表

ーCNNアンサンブル学習に基づく音響識別によるプール沸騰の発生検知ー

2025年7月18日

植木研究室 8125517 柴崎陸

Agenda

01 背景・目的

02 前期に取り組んだこと

- 先週までのまとめ
- 新しくやったこと

03 今後の計画

Agenda

01 背景・目的

02 前期に取り組んだこと

- 先週までのまとめ
- 新しくやったこと

03 今後の計画

研究目的 – 先行研究

先行研究

- プール沸騰時の熱流束を，機械学習モデル(PCA分析，CNN)を使用して高い予測精度(決定係数 97.4%、平均絶対誤差7.0%)を達成[6]
- STFT，CWTを用いたCNNモデルを使用してモデルの判断根拠を可視化 & 熱流束予測時の決定係数 99% を達成[7]



熱流束予測は高精度で可能だが，

早期検知に重要な，**沸騰-未沸騰状態の分類(沸騰開始点の予測)**においては乏しい

実機環境では多くのノイズが発生する



- ✓ **信号対ノイズ比(SNR)**における予測性能の検証
- ✓ **アンサンブル学習**による安定性と精度の向上

研究目的

目的

- ✓ 早期検知に重要な沸騰-未沸騰分類を，高精度な沸騰開始点予測手法を確立し，核沸騰開始点を100% 分類可能な熱流束を予測
- ✓ 実機環境を想定し，SNRを付加したときのノイズ耐性の評価とアンサンブル学習による予測の安定性と精度の向上

研究目的

修士論文の目的

- 特徴抽出手法の最適化
- 最新アーキテクチャによるモデルの高度化
- 多様な条件下での，試構築モデルの汎用性と頑健性の実証

Agenda

01 背景・目的

02 前期に取り組んだこと

- 先週までのまとめ
- 新しくやったこと

03 今後の計画

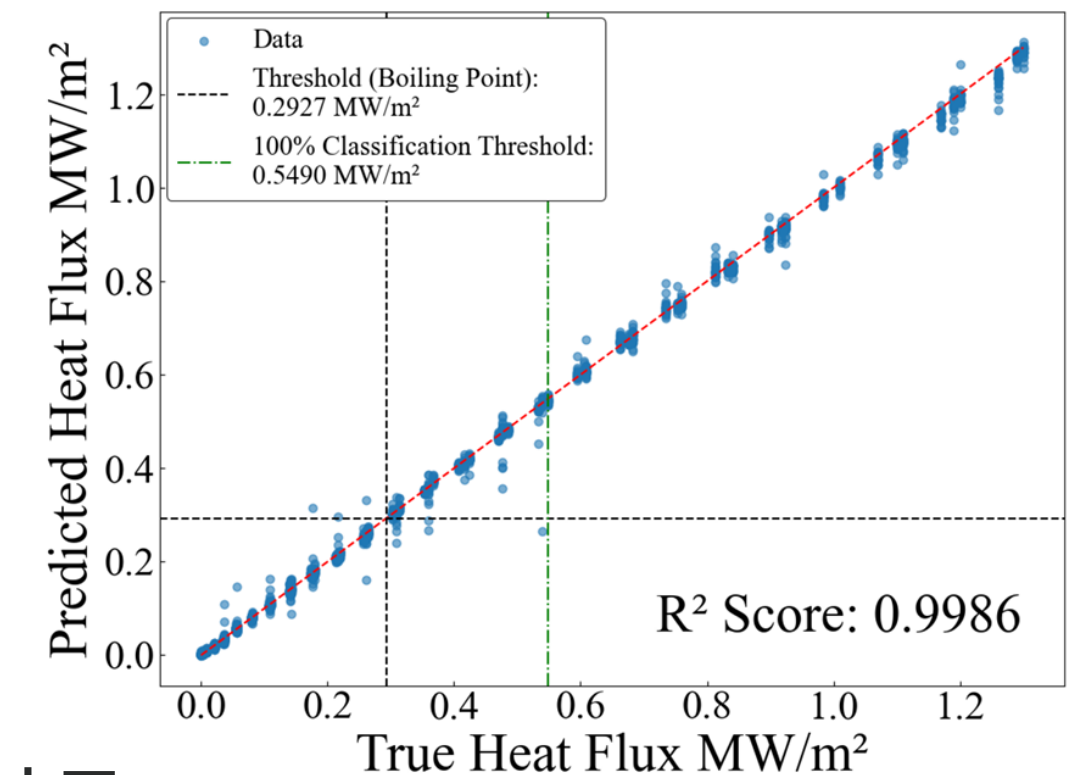
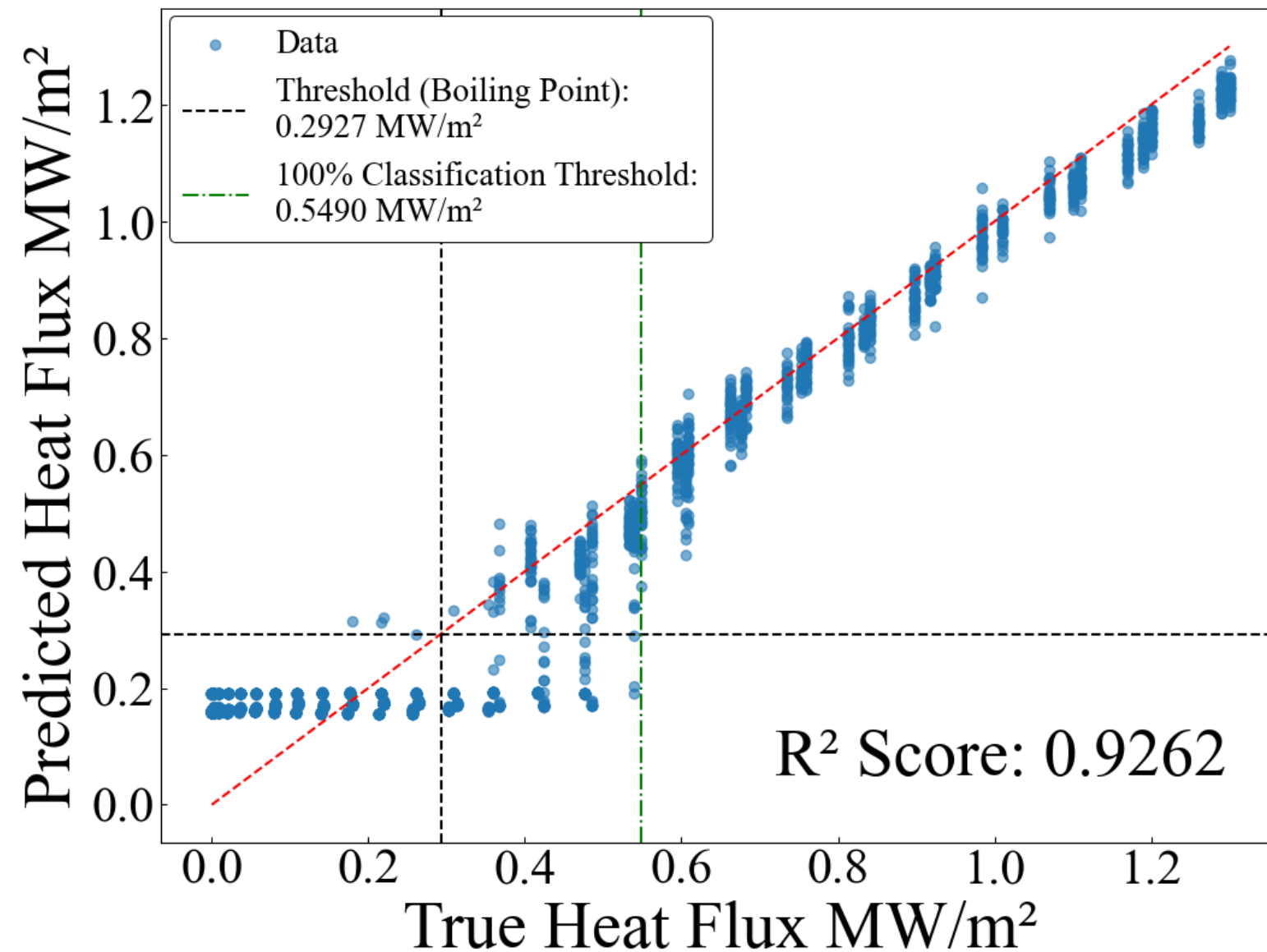
取り組んだこと：先週までのまとめ

AlexNet, ResNet50, VGG16でアンサンブル

STFTの方法を修正し，学習データが変わった



沸騰開始点近傍の予測精度が悪くなったが，音圧が比較的低いところだったので妥当とした



● 卒論時の
ノイズなしの結果

Fig. G.1.1 SNR = no_noise (split=1)

Agenda

01 背景・目的

02 前期に取り組んだこと

- 先週までのまとめ
- 新しくやったこと

03 今後の計画

取り組んだこと：新しくやったこと

試した新しいアーキテクチャ

- MobileNet v2
- EfficientNet b0
- CRNN (CNN+RNN)
 - SELDnet (Sound Event Localization and Detection)
- CNN+Transformer
- AlexNet+Transformer

取り組んだこと：新しくやったこと

試した新しいアーキテクチャ

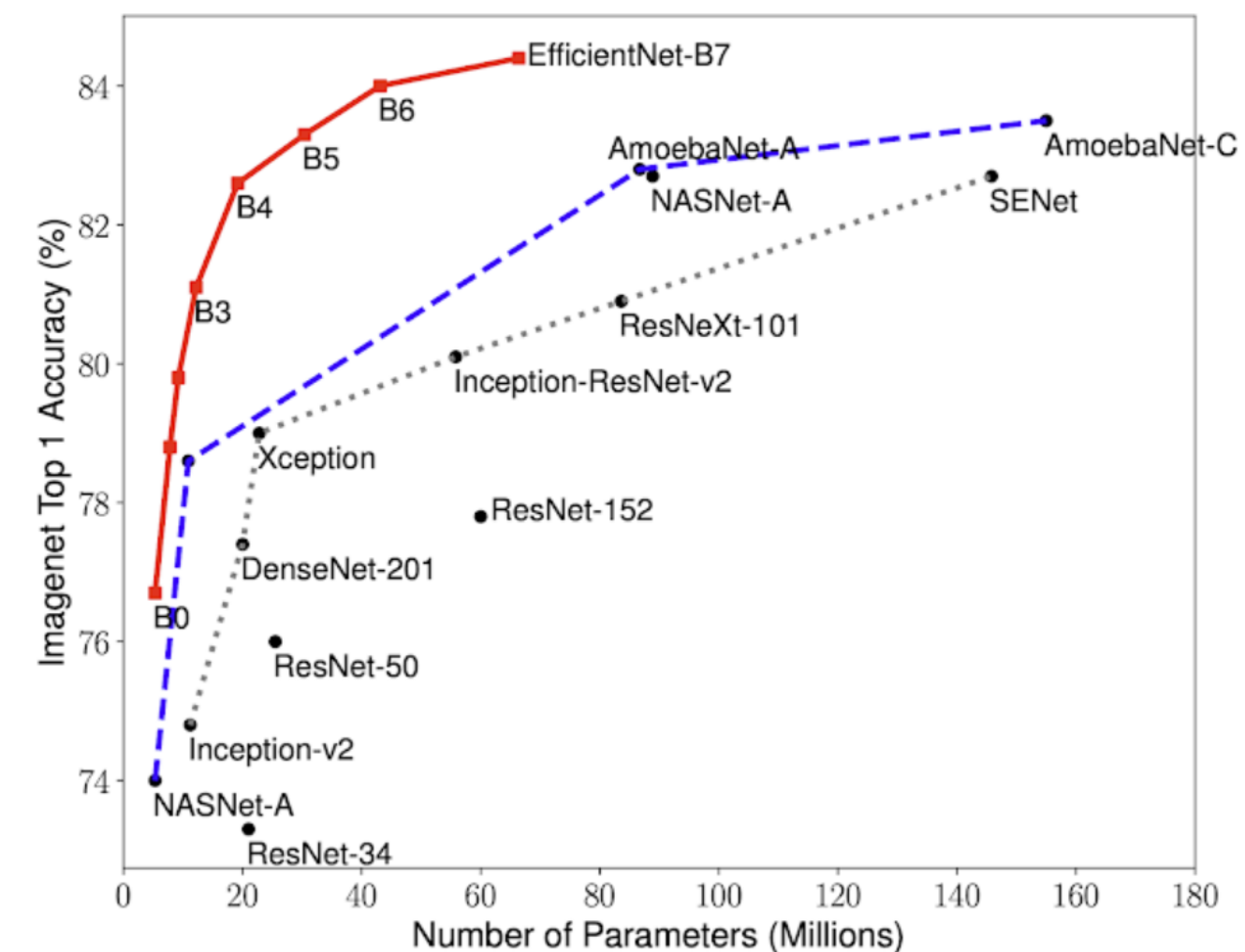
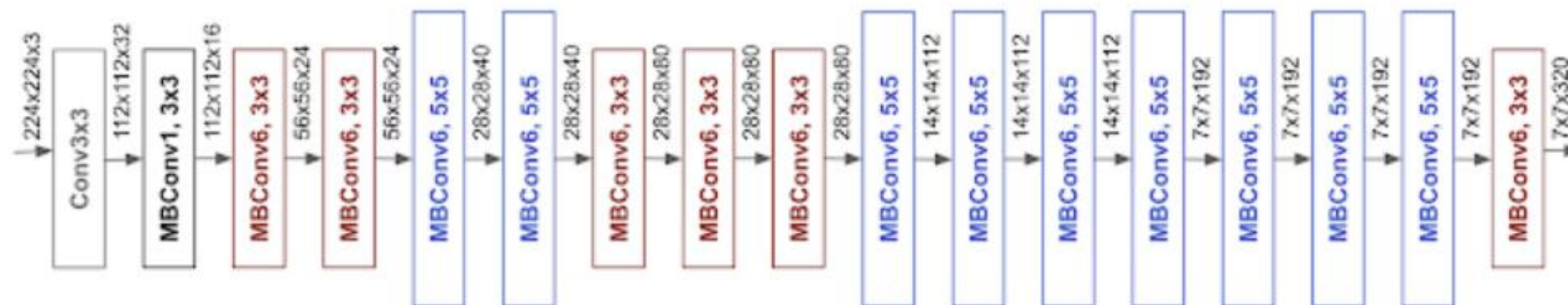
- MobileNet v2
- EfficientNet b0
- CRNN (CNN+RNN)
 - SELDnet (Sound Event Localization and Detection)
- CNN+Transformer
- AlexNet+Transformer

Input	Operator	t	c	n	s
$224^2 \times 3$	conv2d	-	32	1	2
$112^2 \times 32$	bottleneck	1	16	1	1
$112^2 \times 16$	bottleneck	6	24	2	2
$56^2 \times 24$	bottleneck	6	32	3	2
$28^2 \times 32$	bottleneck	6	64	4	2
$14^2 \times 64$	bottleneck	6	96	3	1
$14^2 \times 96$	bottleneck	6	160	3	2
$7^2 \times 160$	bottleneck	6	320	1	1
$7^2 \times 320$	conv2d 1x1	-	1280	1	1
$7^2 \times 1280$	avgpool 7x7	-	-	1	-
$1 \times 1 \times 1280$	conv2d 1x1	-	k	-	-

- Bottleneck：チャンネル方向を拡張→個別畳み込み→チャンネル方向に結合

試した新しいアーキテクチャ

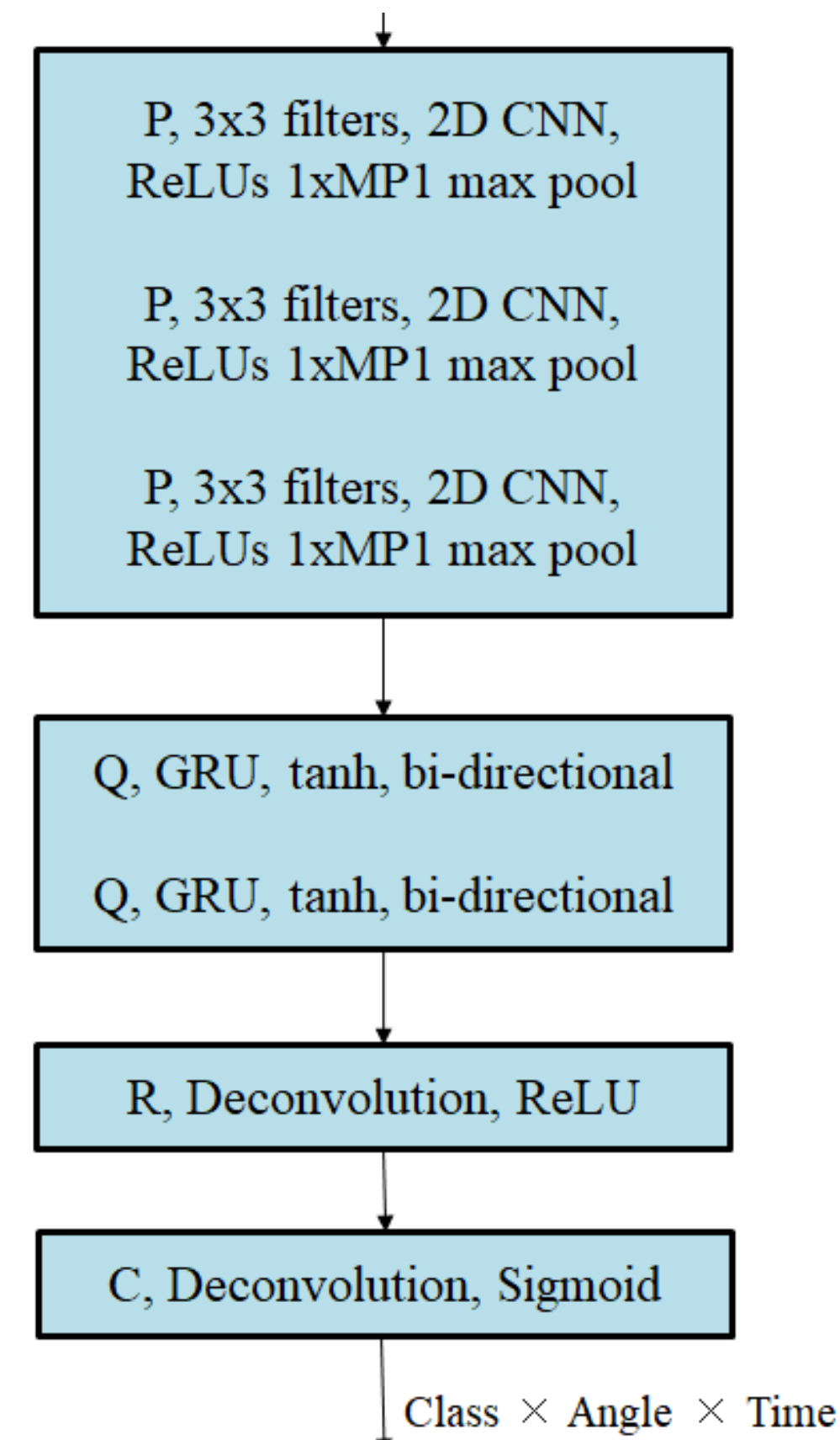
- MobileNet v2
- EfficientNet b0
- CRNN (CNN+RNN)
 - SELDnet (Sound Event Localization and Detection)
- CNN+Transformer
- AlexNet+Transformer



取り組んだこと：新しくやったこと

試した新しいアーキテクチャ

- MobileNet v2
- EfficientNet b0
- CRNN (CNN+RNN)
 - SELDnet (Sound Event Localization and Detection)
- CNN+Transformer
- AlexNet+Transformer
- GRU : Gated Recurrent Unitで,
RNNの一種



取り組んだこと：新しくやったこと

試した新しいアーキテクチャ

- MobileNet v2
- EfficientNet b0
- CRNN (CNN+RNN)
 - SELDnet (Sound Event Localization and Detection)
- CNN+Transformer
- AlexNet+Transformer

1. CNNで特徴抽出
2. Tfで時間分析
3. プーリング

```
def cnn_transformer(
    self,
    pool_sizes=(8, 8, 2),
    num_transformer_blocks=4,
    head_size=256,
    num_heads=4,
    ff_dim=4,
):
    """
    2D-CNN + Transformer Encoder を用いた回帰モデル。
    """
    x_in = Input(shape=self.input_shape, name="spec_input")

    # --- CNN ---
    x = x_in
    for p in pool_sizes:
        x = Conv2D(64, (3, 3), padding="same")(x)
        x = BatchNormalization()(x)
        x = Activation("relu")(x)
        x = MaxPooling2D(pool_size=(1, p))(x)
        x = Dropout(0.2)(x)

    # --- 2D → 3D reshape ---
    T = self.input_shape[0]
    x = Permute((2, 1, 3))(x)
    x = Reshape((T, -1))(x)

    # --- Transformer Encoder ---
    # Transformer Encoder を複数重ねる
    for _ in range(num_transformer_blocks):
        x = transformer_encoder(
            x, head_size, num_heads, ff_dim, dropout=0.2
        )

    # --- 時間平均 → 回帰線形出力 ---
    x = GlobalAveragePooling1D()(x)
    output = Dense(1, activation="linear", name="output")(x)

    return Model(inputs=x_in, outputs=output)
```

取り組んだこと：新しくやったこと

試した新しいアーキテクチャ

- MobileNet v2
- EfficientNet b0
- CRNN (CNN+RNN)
 - SELDnet (Sound Event Localization and Detection)
- CNN+Transformer
- AlexNet+Transformer
- GAP：全ての時間ステップを平等に平均
- ATP：各時間の重要度による加重平均

```
def cnn_transformer_v1(self, num_transformer_blocks=4, head_size=256, num_heads=4, ff_dim=2048):
    """
    over-indented
    AlexNetをCNNベースとして利用し、Transformer Encoderに接続する回帰モデル。
    """
    x_in = Input(shape=self.input_shape, name='spec_input')

    # --- AlexNetベースの特徴抽出器 ---
    # 元のalexnetのConv層とPooling層をFunctional APIで記述
    x = Conv2D(96, (11, 11), strides=(4, 4), activation='relu', padding="same")(x_in) line too long (81
    x = BatchNormalization()(x)
    x = MaxPooling2D(pool_size=(3, 3), strides=(2, 2), padding="same")(x) line too long (81
    x = Conv2D(256, (5, 5), strides=(1, 1), activation='relu', padding="same")(x) line too long (81
    x = BatchNormalization()(x)
    x = MaxPooling2D(pool_size=(3, 3), strides=(2, 2), padding="same")(x) line too long (81
    x = Conv2D(384, (3, 3), strides=(1, 1), activation='relu', padding="same")(x) line too long (81
    x = Conv2D(384, (3, 3), strides=(1, 1), activation='relu', padding="same")(x) line too long (81
    x = Conv2D(256, (3, 3), strides=(1, 1), activation='relu', padding="same")(x) line too long (81
    x = MaxPooling2D(pool_size=(3, 3), strides=(2, 2), padding="same")(x) line too long (81

    # --- 2D->3D reshape (seldnet_regressorと同じ) ---
    shape = x.shape
    x = Reshape((shape[1], shape[2] * shape[3]))(x)

    model_dim = 32 # Transformerが扱うモデル次元
    x = TimeDistributed(Dense(model_dim, activation='relu'))(x)

    x = PositionalEmbedding(sequence_length=shape[1], output_dim=x.shape[-1])(x) line too long (81

    # --- Transformer Encoder ブロック ---
    # Transformer Encoderを複数重ねる
    for _ in range(num_transformer_blocks):
        x = transformer_encoder(x, head_size, num_heads, ff_dim, dropout=0.2) line too long (81

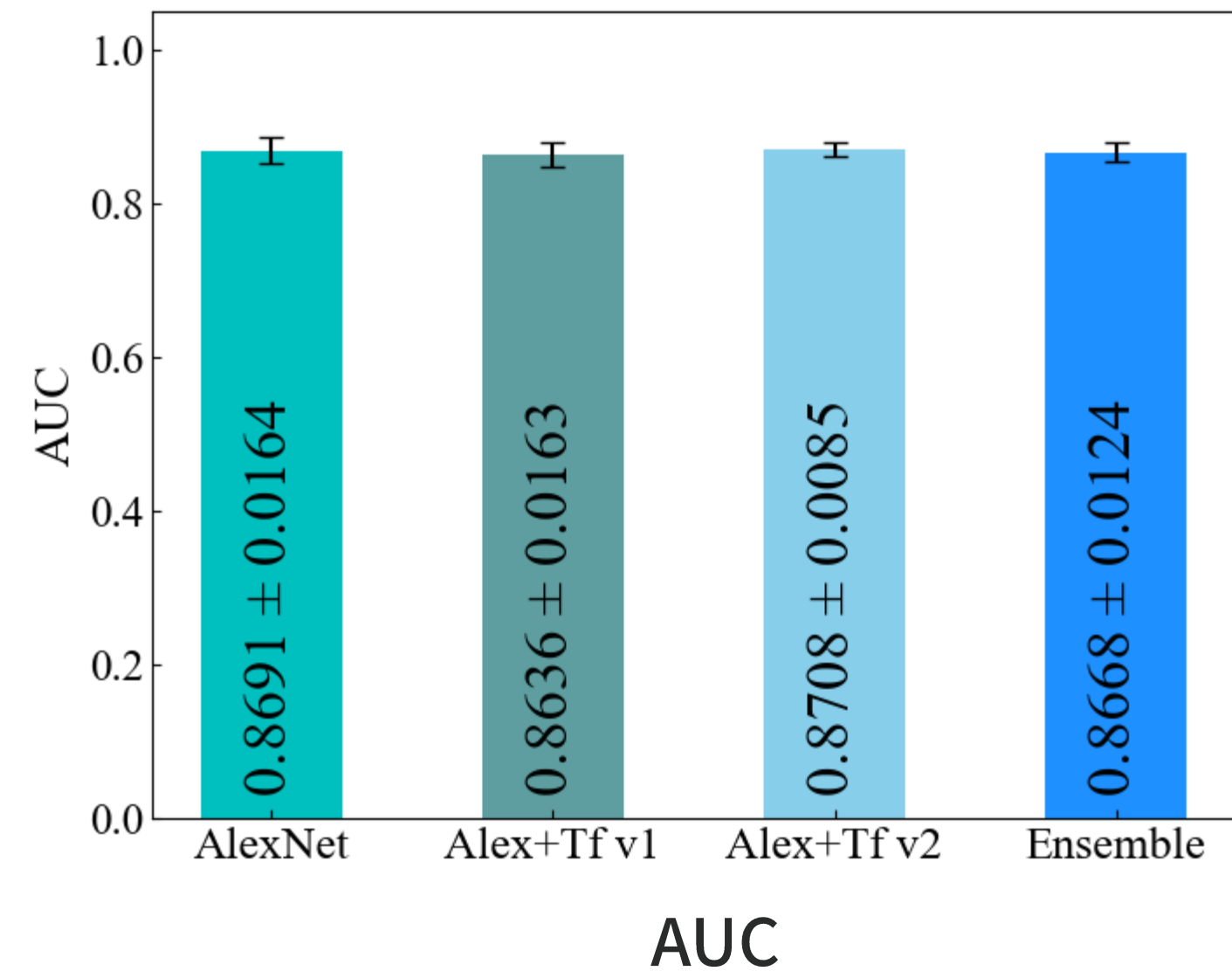
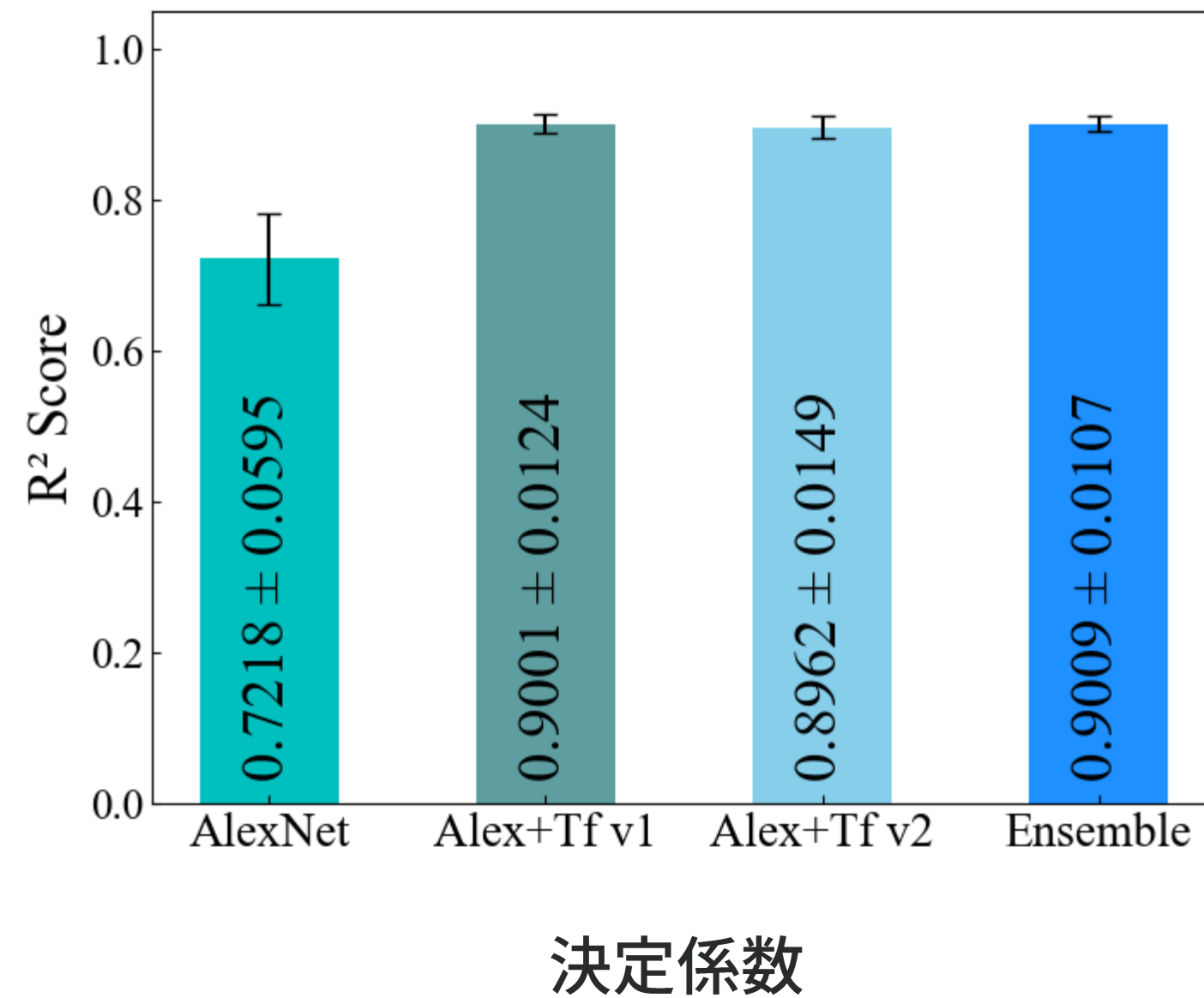
    # --- 時間平均 → 回帰線形出力 (seldnet_regressorと同じ) ---
    # x = GlobalAveragePooling1D()(x)
    x = AttentionPooling()(x)
    output = Dense(1, activation='linear', name='output')(x)

    return Model(inputs=x_in, outputs=output)
```

取り組んだこと：新しくやったこと

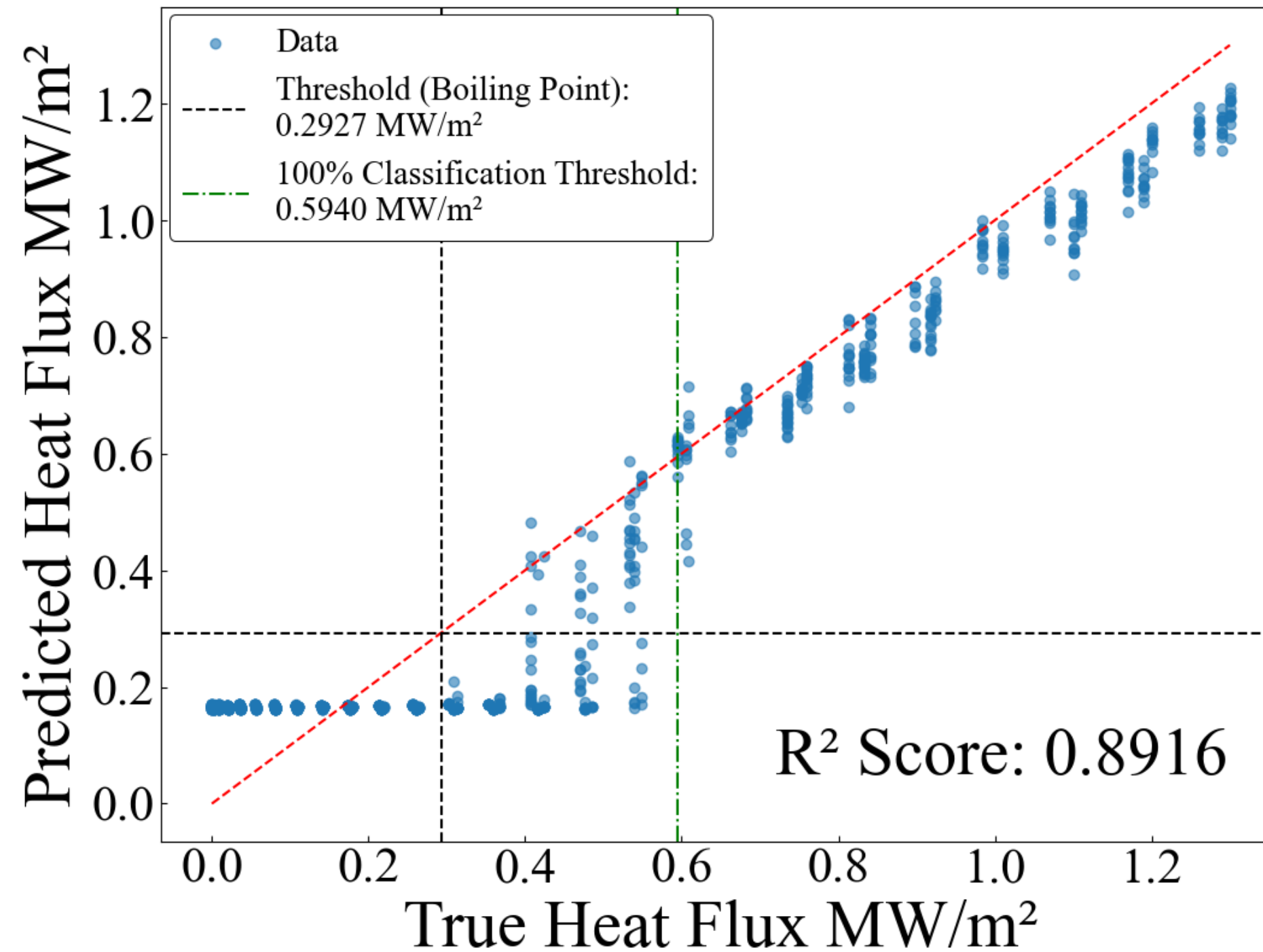
AlexNet, AlexNet+Transformer(GAP), Alex+tf(AttentionPooling)でアンサンブル

- 5分割交差検証の結果．パラメータは個別でチューニングした．



取り組んだこと：新しくやったこと

AlexNet, AlexNet+Transformer(GAP), Alex+tf(AttentionPooling)でアンサンブル



Agenda

01 背景・目的

02 前期に取り組んだこと

- 先週までのまとめ
- 新しくやったこと

03 今後の計画

今後の計画

項目		4月	5月	6月	7月	8月	9月	10月	11月	12月	1月	2月	3月
特徴量抽出手法の比較	CWT, SWT												
	MFCC												
	Wigner-Ville												
	Highpass, noise												
アンサンブル, モデル選定	モデル数選定												
	Efficientnet系												
	Densenet系												
	Transformer系												
	最適特徴量選定												
	Wavenet入れるか否か												
	モデル・特徴量組み合わせ選定												
音響データを直接入力	Wavenet単体で検証												
	Wavenet+stftで検証												
	Wavenet+メルフィルタで検証												

今後の計画

項目		4月	5月	6月	7月	8月	9月	10月	11月	12月	1月	2月	3月
特徴量抽出手法の比較	CWT, SWT												
	MFCC												
	Wigner-Ville												
	Highpass, noise												
アンサンブル, モデル選定	モデル数選定												
	Efficientnet系												
	Densenet系												
	Transformer系												
	最適特徴量選定												
	Wavenet入れるか否か												
	モデル・特徴量組み合わせ選定												
音響データを直接入力	Wavenet単体で検証												
	Wavenet+stftで検証												
	Wavenet+メルフィルタで検証												

今後の計画

最終目標

- 特徴抽出手法の最適化
- 最新アーキテクチャによるモデルの高度化
- 多様な条件下での，試構築モデルの汎用性と頑健性の実証

今後やること

- スペクトログラムの作成方法の検討
- CNN + Transformer
 - 論文探す
 - チューニング
- 新しく実験したデータで検証